

Computational Engineering

universitat politècnica de catalunya

Assignment 1: Potential flow in a channel: Rotating cylinder.

Antonio Luque López

MUEA

22/03/2025

Contents

1 Problem specification **2**
1.1 Potential Flow in a 2D Channel 2
1.2 Potential Flow in a Channel with a Stationary Cylinder 2
1.3 Potential Flow in a Channel with a Rotating Cylinder 2

2 Theoretical approach **2**
2.1 Potential flow 2
2.2 Stream function approach 3

3 Numerical methodology **3**

4 Code structure **4**

5 Code Verification **5**
5.1 Case 1 Verification: Potential Flow in a Channel 5
5.1.1 Analytical Solution 5
5.1.2 Numerical Implementation 6
5.1.3 Results verification 6
5.2 Case 2 Verification: Potential Flow in a Channel with an static cylinder . . 8
5.2.1 Analytical Solution 8
5.2.2 Numerical Implementation 9
5.2.3 Results verification 9
5.2.4 Physical Analysis 11
5.3 Case 3 verification: Potential Flow in a Channel with a rotating cylinder . 11
5.3.1 Analytical solution 12
5.3.2 Numerical Implementation 12
5.4 Results verification 12
5.4.1 Physical Analysis 14

6 Numerical studies **14**
6.1 Mesh study 14
6.2 Evolution of the Normalized Error of v_p as a Function of δ 15

7 Conclusions **16**

8 Annexes **17**

1 Problem specification

The objective of this study is to simulate an incompressible potential flow in a 2D channel and analyze the effects of introducing a solid obstacle (cylinder) and its rotation.

1.1 Potential Flow in a 2D Channel

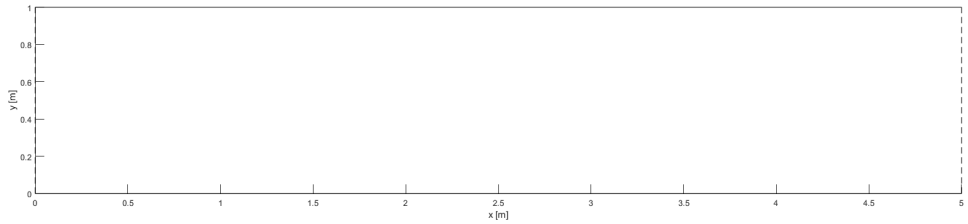


Figure 1: Diagram of the channel case.

1.2 Potential Flow in a Channel with a Stationary Cylinder

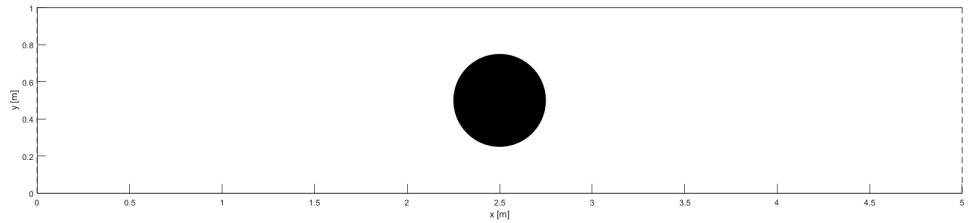


Figure 2: Diagram of the stationary cylinder case.

1.3 Potential Flow in a Channel with a Rotating Cylinder

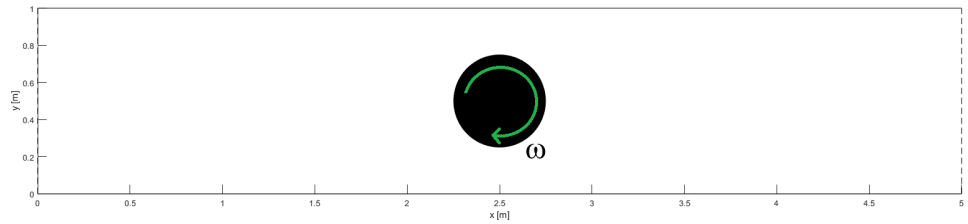


Figure 3: Diagram of the rotating cylinder case.

2 Theoretical approach

2.1 Potential flow

Inviscid flow describes the motion of a fluid far away of the boundary layers, where the effects of friction and heat transfer can be neglected.

$$\tau_{ij} = 0 \tag{1}$$

If we consider only the cases where the flow is irrotational:

$$\nabla \times \overrightarrow{v} = \overrightarrow{w} = 0 \tag{2}$$

Matematically, the flow can be solved by the **stream function approach**, that is only valid for 2D, irrotational (not compulsory) and steady flows.

As a resume:

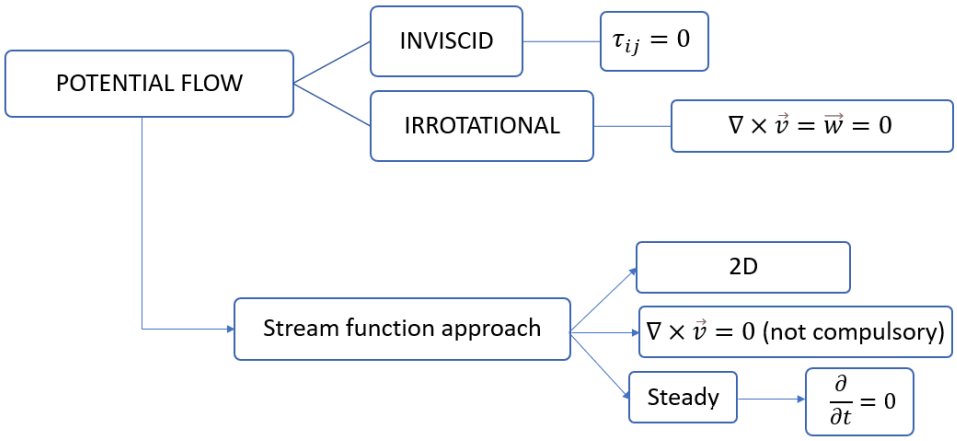


Figure 4: Potential flow in a graphic diagram

2.2 Stream function approach

The stream function is a mathematical expression that describes the motion of a bidimensional and steady flow.

$$v_x = \frac{\rho_{ref}}{\rho}\frac{\partial \psi}{\partial y}, \quad v_y = -\frac{\rho_{ref}}{\rho}\frac{\partial \psi}{\partial x} \tag{3}$$

Applying the irrotationality condition, the following relation can be obtained:

$$\frac{\partial}{\partial x}\left(\frac{\rho_{ref}}{\rho}\frac{\partial \psi}{\partial x}\right) + \frac{\partial}{\partial y}\left(\frac{\rho_{ref}}{\rho}\frac{\partial \psi}{\partial y}\right) = -\omega_z = 0 \tag{4}$$

3 Numerical methodology

Using the Stokes theorem, taking in to account that the flow is irrotational and discretizing for a volume P:

$$\Gamma = \oint_C \vec{v} \cdot d\vec{\ell} \approx v_{y_e}\Delta y_P - v_{xn}\Delta x_P - v_{yw}\Delta y_P + v_{xs}\Delta x_P = 0 \tag{5}$$

Introducing the stream function definition and aproximating the differentials in x and y directions:

$$-\frac{\rho_{ref}}{\rho_e}\frac{\psi_E-\psi_P}{d_{PE}}\Delta y_P-\frac{\rho_{ref}}{\rho_n}\frac{\psi_N-\psi_P}{d_{PN}}\Delta x_P+\frac{\rho_{ref}}{\rho_w}\frac{\psi_P-\psi_W}{d_{PW}}\Delta y_P+\frac{\rho_{ref}}{\rho_s}\frac{\psi_P-\psi_S}{d_{PS}}\Delta x_P=0 \quad (6)$$

Finally, the matricial expression for the discretization equation is:

$$a_P[i][j]\psi[i][j] = a_E[i][j]\psi[i+1][j]+a_W[i][j]\psi[i-1][j]+a_N[i][j]\psi[i][j+1]+a_S[i][j]\psi[i][j-1]+b_P[i][j]$$

4 Code structure

The code is structured into multiple stages, starting with the definition of physical and numerical parameters, including domain geometry, inlet conditions, and fluid properties. The function `mesh_generator` is used to generate the computational grid, dividing the domain into an $N \times M$ mesh. Then, the function `cylinder_generator` introduces a cylinder into the domain, defining which cells belong to the solid and which to the fluid.

Once the domain is set up, the function `compute_coef` calculates the coefficients for the stream function ψ, applying appropriate boundary conditions for the channel and the cylinder. The Laplace equation is solved numerically using the function `gauss_seidel`, which iteratively updates the solution until a predefined convergence criterion is met. Then, the function `compute_velocity` computes the velocity distribution throughout the domain, obtaining the velocity components v_x, v_y, and the magnitude v_P. Afterward, the function `compute_thermo_properties` determines thermodynamic properties, such as pressure p_P, temperature T_P, and density ρ_P, assuming the fluid behaves as an ideal gas. After that, the function `final_calculations` is used for computing different parameters as the circulation and the aerodynamic coefficients.

Finally, the code visualizes the results using `plotCFD`, which generates a colormap of the velocity magnitude across the domain, while the `fill` function renders the cylinder. Other functions, such as `plotStreamLines` and `mesh_plot`, though commented out, would allow for the visualization of streamlines and the computational mesh, respectively. This approach provides a detailed numerical solution of the flow around the cylinder, facilitating the analysis of velocity distribution, the cylinder's influence on the flow, and variations in thermodynamic properties.

***For further explanations, refer to the code, where all functions are documented in detail.**

Code instructions

- **CASE**: Selects the simulation scenario.
  - `CASE = 1`: Channel without cylinder.
  - `CASE = 2`: Channel with a static cylinder.
  - `CASE = 3`: Channel with a rotating cylinder (clockwise rotation, $\omega > 0$).
- **SLINES**: Controls the display of streamlines.
  - `SLINES = 1`: Plot streamlines (only works with v_P, v_{xP}, and v_{yP}).
  - `SLINES = 0`: Do not plot streamlines.
- **MESH**: Controls mesh visualization.
  - `MESH = 1`: Plot the computational mesh (not recommended for final figures).
  - `MESH = 0`: Do not plot the mesh.
- **PLOT**: Selects which field to visualize as a color map.
  - `PLOT = 1`: Magnitude of velocity v_P.
  - `PLOT = 2`: Horizontal velocity component v_{xP}.
  - `PLOT = 3`: Vertical velocity component v_{yP}.
  - `PLOT = 4`: Pressure field p_P.
  - `PLOT = 5`: Temperature field T_P.

These parameters allow the user to configure the physical case and customize the graphical output in a simple and modular way.

5 Code Verification

5.1 Case 1 Verification: Potential Flow in a Channel

5.1.1 Analytical Solution

For an inviscid, incompressible, and irrotational flow in a 2D channel, the analytical solution for a uniform flow of velocity U in the x-direction is:

$$\psi(x, y) = -Vx + Uy \tag{7}$$

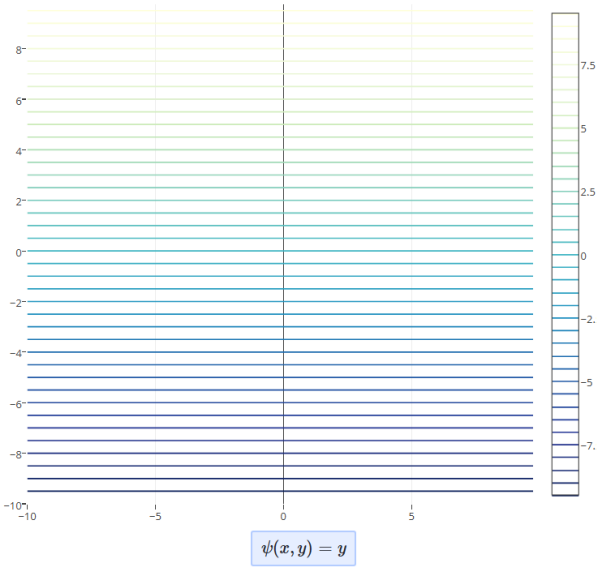


Figure 5: Analytical solution for a channel flow. V=0 m/s, U=1 m/s

5.1.2 Numerical Implementation

The stream function is computed numerically by solving the discretized stream function in all the fluid domain for x and y dymensions:

The following boundary conditions are applied:

- **Inlet** $(x = 0)$: $\psi(0, y) = Uy$.
- **Outlet** $(x = L)$: Neumann condition $\frac{\partial \psi}{\partial x} = 0$.
- **Walls** $(y = 0$ **and** $y = H)$: $\psi = 0$, enforcing the no-penetration condition.

The system of equations is solved iteratively using the Gauss-Seidel method until convergence is reached.

5.1.3 Results verification

For this case, the results presented consider a channel with dimensions $L \times H = 5 \times 1$ m, an inlet velocity profile of 1 m/s, a temperature of 300 K, and a pressure of 101325 Pa.

The mesh, created can be seen in the following graphic:

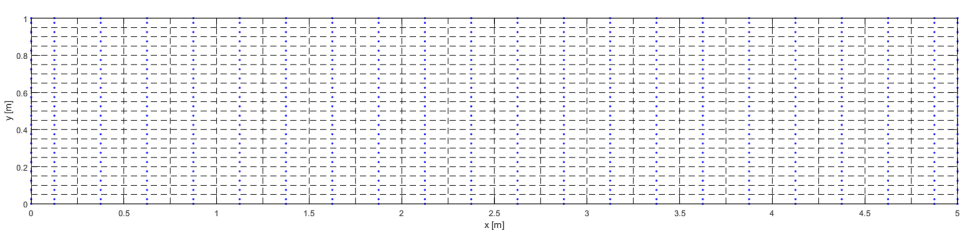


Figure 6: Mesh created. N=20, M=20

If v_P is plotted, it can be observed that its values are almost identical to v_{xp}. This confirms that the velocity field is predominantly aligned with the x-direction. Additionally, when comparing with v_{yp}, it can be seen that $v_{yp} = 0$, which is consistent with the expected results for this case.

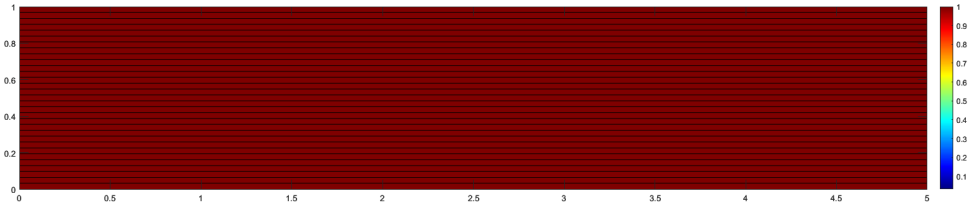


Figure 7: Velocity magnitude v_{xP}

Figure 8 shows a comparison between v_{xp} and v_{yp}, illustrating that the flow is purely horizontal, while Figure 7 presents the overall velocity magnitude v_P.

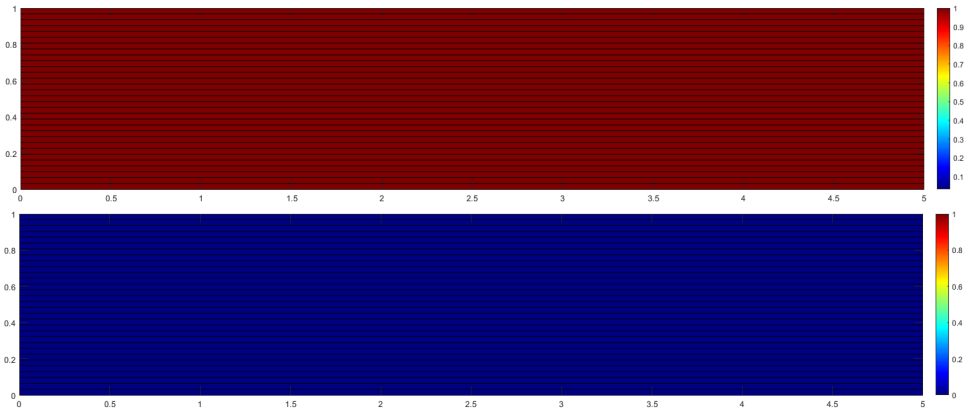


Figure 8: Velocity components v_{xP}(up) and v_{yP}(down)

Finally, the field pressure and temperature are plotted:

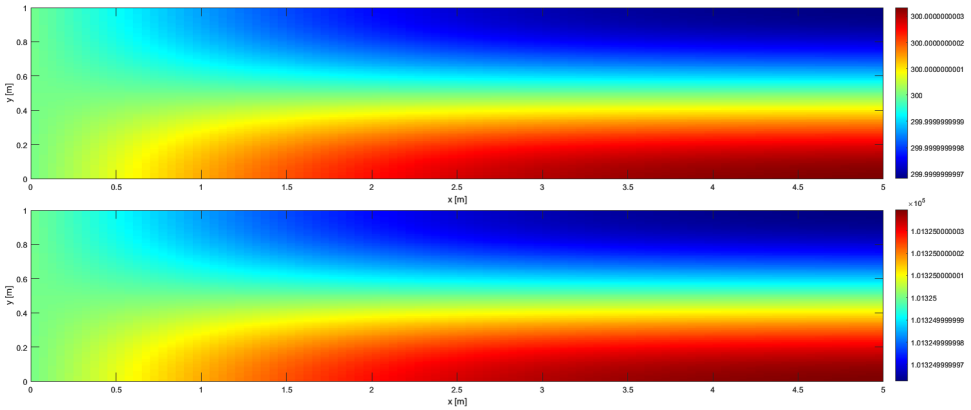


Figure 9: Temperature(K) (top) and pressure (Pa) (bottom) fields of a channel flow.

The pressure and temperature fields appear significantly different because the streamlines are not included in these plots. However, as shown in the following figure, the difference

between the highest and lowest values in both the pressure and temperature fields is minimal, so it can be considered that these fields remain practically constant throughout the domain.

5.2 Case 2 Verification: Potential Flow in a Channel with an static cylinder

For this case, the results presented consider a channel with dimensions $L \times H = 2 \times 1$ m, an inlet velocity profile of 1 m/s, a temperature of 300 K, and a pressure of 101325 Pa. Also, the cylinder is located in (L/2, H/2), with a radious of $R = 0.1$ m

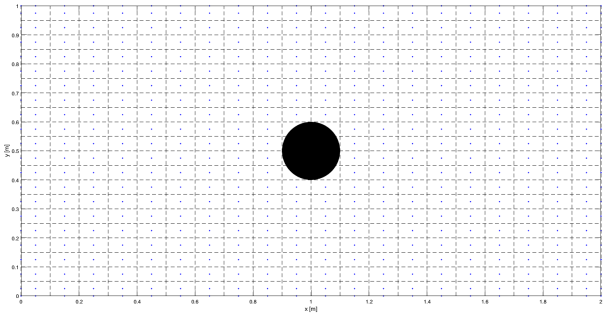


Figure 10: Mesh created with a cylinder. N=20, M=20

5.2.1 Analytical Solution

In a potential flow around a stationary cylinder, the stream function can be expressed as:

$$\psi(x,y) = -Vx + Uy + \frac{\mu}{2\pi} \cdot \frac{(y-y_0)\cos(\alpha) - (x-x_0)\sin(\alpha)}{(x-x_0)^2 + (y-y_0)^2} \tag{8}$$

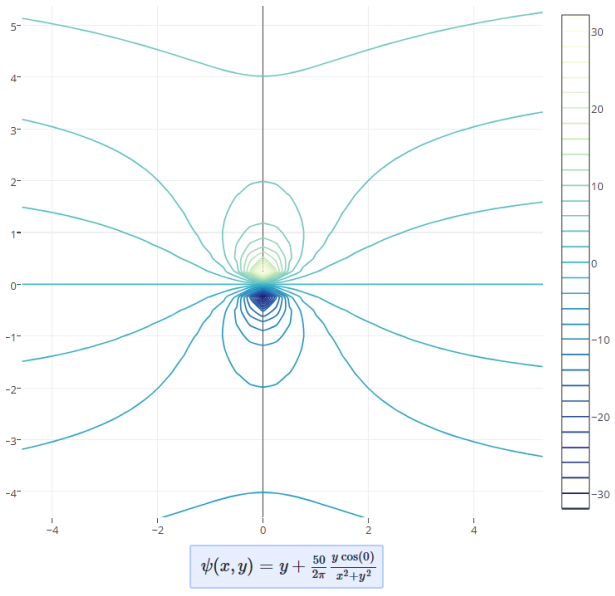


Figure 11: Analytical solution for an static cylinder.

This expression (Uniform+Dipole) and the representation in xy coordinates, show that the flow is symmetric and satisfies the boundary condition at the cylinder surface.

5.2.2 Numerical Implementation

In order to numerically simulate the potential flow around a stationary cylinder, a structured Cartesian grid is used to discretize the computational domain.

First, a set of grid points defining the exterior of the cylinder is created. The cylinder is centered at (x_c, y_c) with a given radius R. For each computational node (x_P, y_P), the radial distance from the cylinder center is computed as:

$$r_P = \sqrt{(x_P - x_c)^2 + (y_P - y_c)^2}. \tag{9}$$

If $r_P < R$, the node is identified as being inside the cylinder. In this case, the numerical implementation enforces the conditions:

$$\psi_P = \psi_{\text{body}}, \quad \rho_P = 0. \tag{10}$$

These conditions ensure that the stream function inside the solid region is set to a constant value $\psi_{\text{body}} = v_{in}(H/2)$, and the density is set to zero, reflecting the fact that the flow cannot exist within the solid boundary.

Additionally, the coefficients of the numerical scheme for nodes inside the cylinder must be modified. The system of equations used for solving the stream function and related properties is adjusted such that the equations for the interior points do not contribute to the solution, effectively treating them as part of the boundary. This is done by setting:

$$a_P = 1, \quad a_E = a_W = a_N = a_S = 0, \quad b_P = \psi_{\text{body}}. \tag{11}$$

This formulation ensures that the interior nodes of the cylinder do not affect the solution of the external flow while correctly imposing the boundary conditions. The exterior flow is then solved iteratively using the chosen numerical scheme, taking into account the modified coefficients.

5.2.3 Results verification

Figure 12 illustrates the velocity field around a stationary cylinder in a potential flow. As expected, the velocity is uniform far from the cylinder, indicating that the free-stream condition is correctly imposed. However, near the cylinder, the velocity distribution is altered due to the presence of the solid.

At the stagnation points, located at the front and rear of the cylinder along the x-axis, the velocity reaches zero. Conversely, along the sides of the cylinder (top and bottom), the velocity reaches its maximum, consistent with the theoretical solution, where the velocity

magnitude at these points is twice the free-stream velocity. Additionally, the symmetry of the streamlines confirms that the flow remains irrotational and inviscid, as expected from potential flow theory.

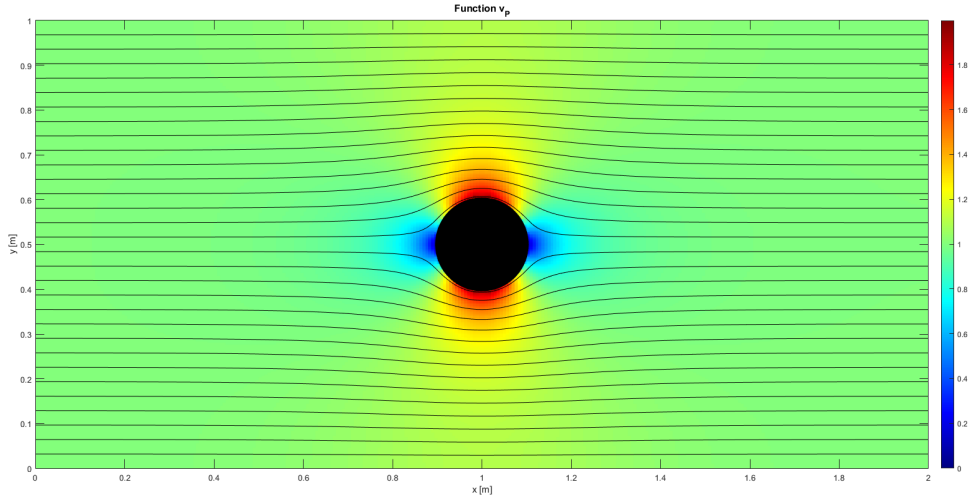


Figure 12: Velocity field around a stationary cylinder in potential flow. The color scale represents the velocity magnitude, in m/s and the black lines indicate streamlines.

Figure 13 shows a comparison between the velocity components v_{xP} and v_{yP} around the cylinder. The horizontal component v_{xP} shows a decrease in velocity near the cylinder due to the presence of the stagnation points, while the vertical component v_{yP} exhibits symmetrical patterns around the cylinder.

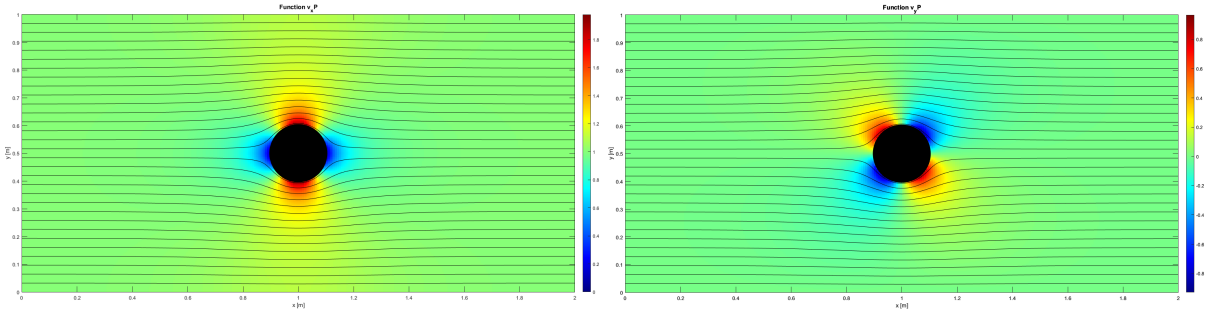


Figure 13: Velocity components v_{xP} (top) and v_{yP} (bottom) around the cylinder.

Finally, the field pressure and temperature are plotted:

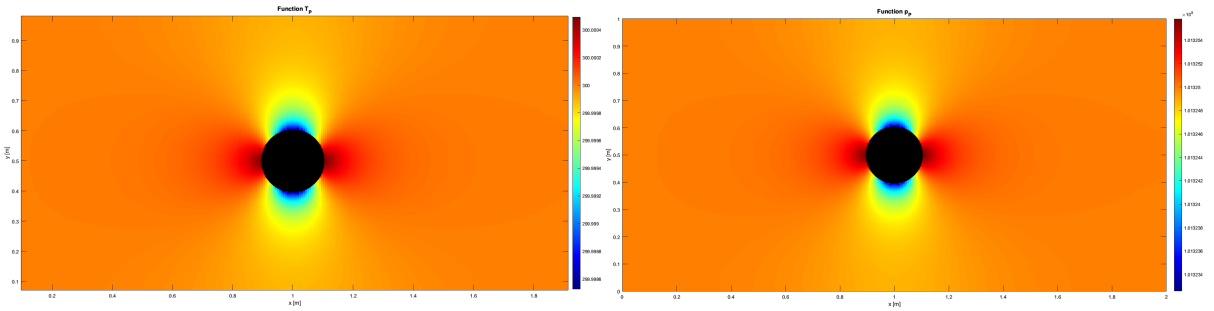


Figure 14: Temperature(K) (left) and pression(Pa) (right) field around the cylinder.

The minimal temperature variation around the cylinder is expected in potential flow since it is inviscid and irrotational, meaning there is no heat transfer due to viscosity. In this idealized case, temperature changes only due to pressure variations.

5.2.4 Physical Analysis

In this section, key aerodynamic parameters around a stationary cylinder are evaluated: circulation, drag, lift, and the pressure coefficient. The objective is to compare the theoretical predictions with the numerical results.

Circulation

For a stationary cylinder in an inviscid, irrotational flow, the circulation is zero, becouse angular velocity (ω) is also zero:

$$\Gamma = 2\pi R^2 \omega = 0. \tag{12}$$

Drag and Lift Forces

For a stationary cylinder in inviscid flow, the theoretical drag is zero, as stated by *d'Alembert's paradox*, and no lift is generated. Compared with the code, the values are almost the same. For N=M=300:

$$L = C_L = -1.6091 \times 10^{-6} \approx 0, \quad D = C_D = -2.9430 \times 10^{-6} \approx 0. \tag{13}$$

5.3 Case 3 verification: Potential Flow in a Channel with a rotating cylinder

For this case, the results presented consider a channel with dimensions $L \times H = 2 \times 1$ m, an inlet velocity profile of 1 m/s, a temperature of 300 K, and a pressure of 101325 Pa. Also, the cylinder is located at $\left(\frac{L}{2}, \frac{H}{2}\right)$, with a radius of $R = 0.1$ m and an angular velocity of $\omega = 25$ rad/s.

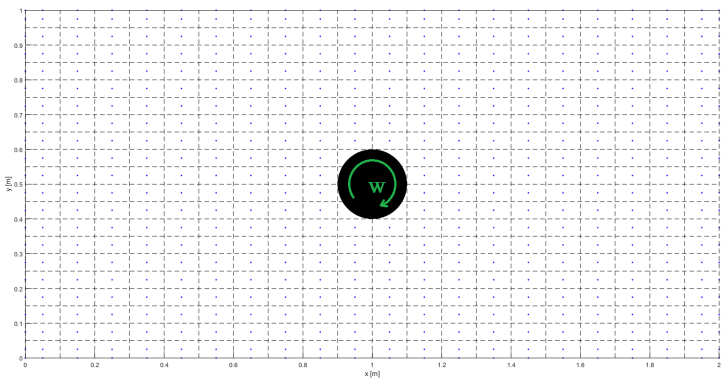


Figure 15: Mesh created with a rotating cylinder. N=20, M=20

5.3.1 Analytical solution

For an inviscid, incompressible, and irrotational flow around a rotating cylinder, the stream function can be expressed as:

$$\psi(x,y) = \psi_{Static\ cylinder} + -\frac{\Gamma}{2\pi}\ln\left(\sqrt{(x-x_0)^2+(y-y_0)^2}\right) \tag{14}$$

Where Γ is the circulation along the cylinder

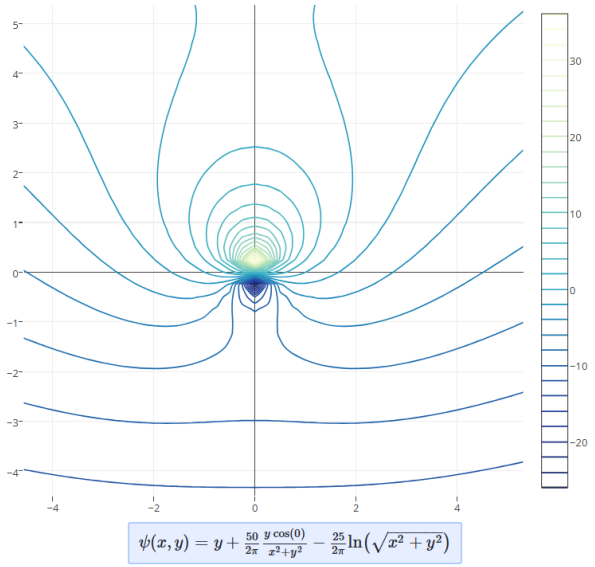


Figure 16: Stream function for a rotating cylinder. $\Gamma = 20\ m^2/s$, $\mu = 50\ m^3/s$

5.3.2 Numerical Implementation

For a rotating cylinder, the function ψ_{body} is modified to account for the induced circulation due to the cylinder's angular velocity ω. The new expression for the stream function inside the solid becomes:

$$\psi_{\text{body}} = v_{\text{in}}\left(\frac{H}{2}\right) + R_{\text{cyl}}^2\omega. \tag{15}$$

Additionally, the coefficients of the numerical scheme for nodes inside the cylinder must be modified. The system of equations used for solving the stream function and related properties is the same that in the previous case, but with the new ψ_{body}

5.4 Results verification

The velocity field around a rotating cylinder in a uniform potential flow is shown in Figure 17. It can be observed that, far from the cylinder, the velocity remains practically constant, however, near the surface of the cylinder, the velocity undergoes significant changes due to the combination of uniform flow and the circulation induced by rotation.

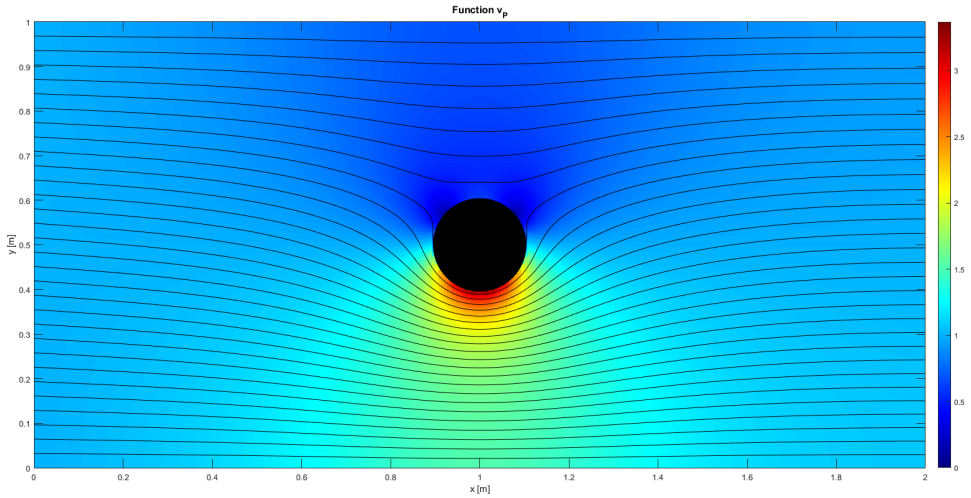


Figure 17: Velocity magnitude v_P (m/s) along a rotating cylinder

While in a stationary cylinder the stagnation points are located at the front and rear of the cylinder, in the rotating case, these points shift, generating an asymmetry in the velocity distribution. The maximum velocity is found on the side where the cylinder's rotation adds to the free-stream velocity, while the minimum velocity occurs on the opposite side, where the rotation opposes the free-stream motion.

Additionally, Figure 18 presents the velocity components in the studied domain.

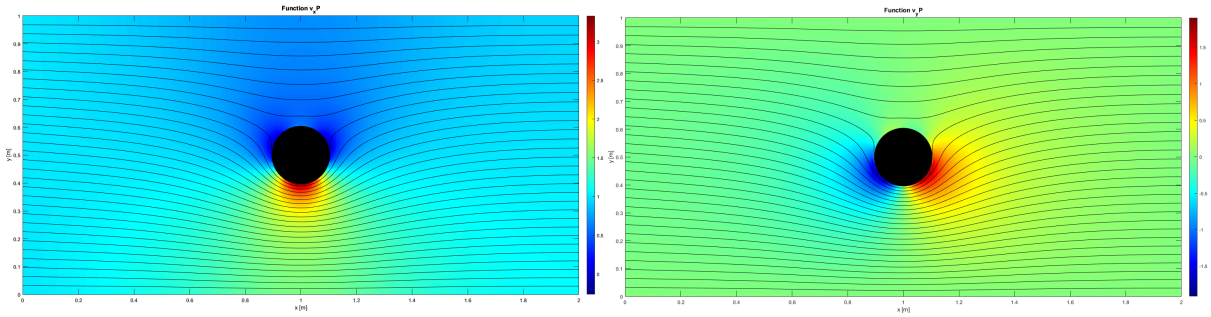


Figure 18: Components of the velocity v_{xP} (top) and v_{yP}(bottom) along a rotating cylinder

The temperature and pressure distributions around the rotating cylinder are shown in Figure 19. Since potential flow is incompressible and irrotational, temperature variations are minimal, solely caused by pressure changes induced by the cylinder's rotation. As expected, pressure decreases in high-velocity regions and increases where velocity is lower. This confirms the presence of circulation and the resulting lift force due to rotation. The pressure asymmetry directly generates lift, explaining how a rotating cylinder deflects the flow and produces lateral forces.

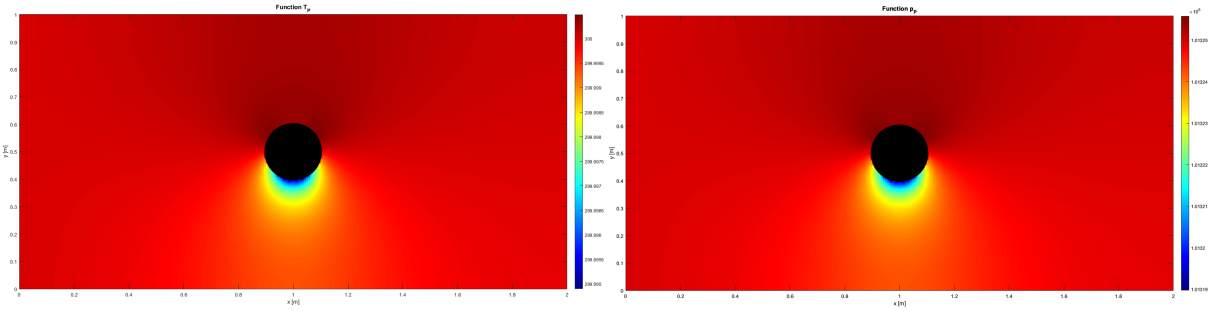


Figure 19: Temperature field(K) (left) and Pressure field (Pa) (right) along a rotating cylinder

5.4.1 Physical Analysis

In this section, key aerodynamic parameters around a rotating cylinder are evaluated: circulation, drag and lift. The objective is to compare the theoretical predictions with the numerical results.

Circulation

For $\omega = 25\,\text{rad/s}$ and $R = 0.1\,\text{m}$, the theoretical circulation in a rotating cylinder along a potential flow is:

$$\Gamma = 2\pi R^2 \omega = 1.571\,\text{m}^2/\text{s}. \tag{16}$$

Drag and Lift Forces

The lift force generated by the rotating cylinder is explained by the *Magnus effect*, which arises due to the circulation induced by the rotation. The drag remains nearly zero, as expected in potential flow. For $N = M = 300$, the computed aerodynamic coefficients are:

$$C_L = -15.7063 \approx -\frac{\Gamma}{v_\infty R_{cyl}} = -15.71, \quad C_D = 0.0083 \approx 0. \tag{17}$$

The results show a significant lift due to the rotational motion, while drag remains negligible, consistent with theoretical expectations in potential flow.

6 Numerical studies

6.1 Mesh study

This section analyzes the error of the numerical solution as a function of the number of discretizations in the mesh. To do so, the case of a stationary cylinder has been considered, and the numerically computed velocity has been compared with its theoretical solution.

To quantify this error, both fields, theoretical and numerical has been subtracted. Additionally, a filtering process was applied to remove values with excessively high errors compared to the mean.

Figure 20 presents the evolution of the error in velocity v_P as a function of the number of discretizations in the mesh. To simplify the analysis, the case where $M = N$ has been considered, ensuring that the mesh resolution increases uniformly in both directions.

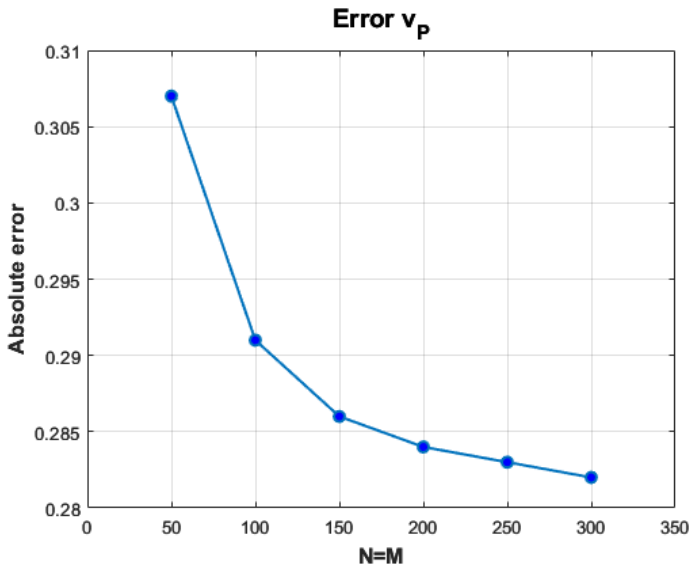


Figure 20: Error in velocity v_P as a function of the number of discretizations.

As observed in the graph, as the number of discretizations increases, the error progressively decreases, confirming the convergence of the numerical solution toward the theoretical one.

6.2 Evolution of the Normalized Error of v_p as a Function of δ

In this section, the evolution of the normalized error of v_p as a function of δ is analyzed for different values of N, M. To facilitate comparison between cases, each set of values has been normalized by dividing by the maximum value of its respective vector. This ensures that the error in each case is bounded between 0 and 1, allowing for a clearer interpretation of the relative reduction in error as δ increases.

The results indicate that the error is larger for small values of δ, but it decreases rapidly, reaching a regime where further improvements become marginal. Additionally, it is observed that for larger N, M, the error decreases across the entire range of δ, suggesting that a finer discretization improves the accuracy of the method.

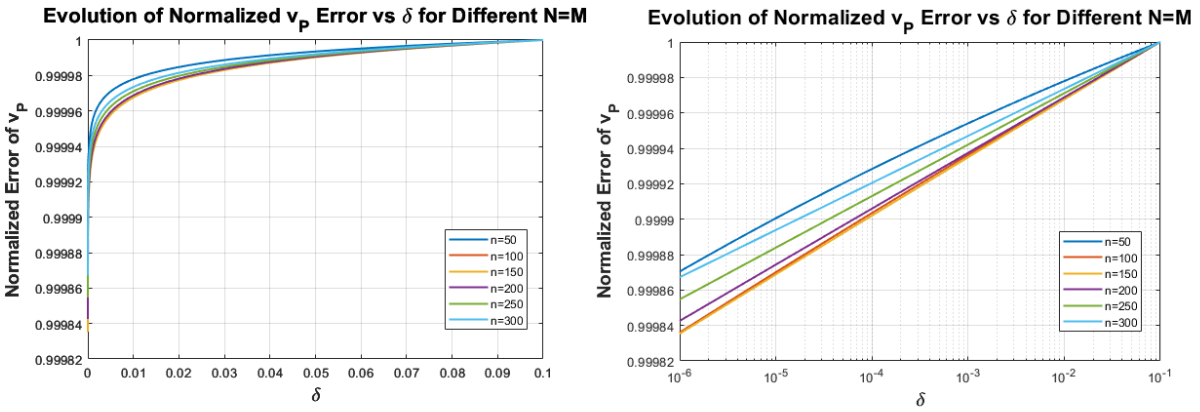


Figure 21: Evolution of the normalized error of V_p as a function of δ for different values of N. Left: Linear scale, Right: Log-scale.

Figure 21 illustrates this evolution using a logarithmic scale on the δ axis, allowing for a clearer visualization of the error trend. It is concluded that to minimize the normalized error, it is crucial to select an appropriate δ and balance N to optimize accuracy while avoiding excessive computational costs.

7 Conclusions

The study conducted has made it possible to simulate and analyze the behavior of potential flow around a cylinder, both stationary and rotating. The results obtained are consistent with classical theory, showing the expected effects such as the symmetry of the velocity field in the stationary case and the appearance of the Magnus effect in the rotating case.

The numerical analysis demonstrated good convergence as the spatial discretization was refined, and the comparison with theoretical solutions helped validate the code implementation.

8 Annexes

```

% ANTONIO LUQUE LOPEZ
% MUEA
% ASSIGNMENT 1: POTENTIAL FLOW IN A CHANNEL
%-------------------------------------------------------
clear workspace,
clear all;
clc;
clearvars;
close all;

%-------------------------------------------------------

%%INSTRUCTIONS%%

%CASE=1 for channel; CASE=2 for channel+static cylinder; CASE=3 for channel+rotating cylinder(w>0-->Clockwise)
%SLINES=1 Plot streamlines(ONLY FOR v_P, v_xP, v_yP); SLINES=0 Don't plot streamlines
%MESH=1 -->Plot the mesh(NOT RECOMENDED); MESH=0 -->Don't plot the mesh
%PLOT=1 for v_P, 2 for v_xP, 3 for v_yP, 4 for p_P, 5 for T_P-->Plot the variable field in a colormap.

CASE=2; SLINES=1; MESH=0; PLOT=1;

%-------------------------------------------------------

% 1. INPUT DATA
% 1.1 Physical data
L=2;
H=1;
v_in=1;
p_in=101325;
T_in=300;

gamma = 1.4;
R = 287;

% 1.2 Numerical data

N = 150;
M = 150;
delta = 1e-10;
psi_start = 0;

% 1.2 Reference data
```

```
44
45 p_ref=101325;
46 T_ref=300;
47 v_ref=1;
48 rho_F = p_in / (R * T_in);
49 rho_ref=rho_F;
50
51 % 1.3 Cylinder data
52
53 if CASE==3
54     w=25; % Enter an angular velocity
55 else
56     w=0;
57 end
58
59 if CASE~=1
60     R_cyl=0.1;
61     x_center=L/2;
62     y_center=H/2;
63     theta=0;
64     psi_body = v_in*(H/2)+R_cyl^2*w;% Valid for static and rotating
          cylinder
65 else
66     R_cyl=0;
67     x_cyl=0;
68     x_center=0;
69     y_center=0;
70     psi_body =0;
71 end
72
73 % 2. PREVIOUS CALCULATIONS
74
75 dx = L/N;
76 dy = H/M;
77 [x_P, y_P] = mesh_generator(N,M,L,H); % Function that creates the
     geometry of the mesh.
78
79 % 3.INITIAL ESTIMATED VALUES
80 psi = psi_start * ones(M+2, N+2);
81 solid = ones(M+2, N+2); %1 for fluid, 0 for solid.
82
83 if CASE~=1
84  [x_cyl, y_cyl, solid, theta] = cylinder_generator(x_center, y_center,
     R_cyl, M, N, x_P, y_P, solid); %Function that creates the cylinder.
85 end
86
87 rho = rho_F*solid; %Density rho_F for fluid, 0 for solid.
88
```

```
% 4. EVALUATE THE DISCRETIZATION COEFFICIENTS
[a_E, a_W, a_N, a_S, b_P, a_P] = compute_coef(psi_start, psi_body, M, N,
    L, H, v_in, y_P, x_P, solid);

% 5. SOLVE THE SET OF DISCRETIZED EQUATIONS
[psi, iter, error] = gauss_seidel(psi, psi_body, solid, a_E, a_W, a_N,
   a_S, a_P, b_P, v_in, y_P, M, N, H, delta);

% 6. OBTAIN THE VELOCITIES AT THE FACES AND THE NODES
[v_xP, v_yP, v_P] = compute_velocity(psi, y_P, x_P, M, N, v_in, rho_ref,
    rho);

% 7. COMPUTE THERMODYNAMICS PROPERTIES
[T_P, p_P, rho_P, cp_mean] = compute_thermo_properties(T_ref, p_ref,
   v_ref, v_P, R, gamma, M, N);

% 8. FINAL CALCULATIONS
if CASE~=1
    [C_D, C_L, Gamma, D, L] = final_calculations(x_P, y_P, p_P, v_xP,
       v_yP, x_cyl, y_cyl, theta, R_cyl, w, rho_ref, v_in);
end
% 9. PRINT RESULTS

switch PLOT
    case 1
        field = v_P;
        title = 'v_P';
    case 2
        fieldToPlot = v_xP;
        title = 'v_xP';
    case 3
        field = v_yP;
        title = 'v_yP';
    case 4
        field = p_P;
        title = 'p_P';
    case 5
        field = T_P;
        title = 'T_P';

end

plotCFD(title, field, N, M, dx, dy);
hold on;

if SLINES==1 % Print the streamlines.
plotStreamLines(psi, x_P, y_P)
hold on
```

```
end

if MESH==1 % Print the mesh.
mesh_plot(x_P, y_P, L, H, N, M)
hold on
end

if CASE~=1 %Print the cylinder
    fill(x_center + (R_cyl+0.7*dx) * cos(linspace(0, 2*pi, 100)),
       y_center + (R_cyl+0.7*dx) * sin(linspace(0, 2*pi, 100)), 'k', '
       FaceAlpha', 1, 'EdgeColor', 'none');
end
```

```

function [a_E, a_W, a_N, a_S, b_P, a_P] = compute_coef(psi_start,
   psi_body, M, N, L, H, v_in, y_P, x_P, solid)

% This function constructs the coefficient matrices required for solving
    the discretized
% form of the equation governing the stream function in a potential flow
   .
%
% INPUTS:
%   psi_start - Initial value of the stream function at the bottom
   boundary.
%   psi_body  - Stream function value imposed on solid boundaries.
%   M         - Number of grid points in the vertical direction.
%   N         - Number of grid points in the horizontal direction.
%   L         - Length of the computational domain.
%   H         - Height of the computational domain.
%   v_in      - Free-stream velocity.
%   y_P, x_P  - Mesh grid points in the y and x directions.
%   solid     - Matrix indicating solid regions (0 for solid, 1 for
   fluid).
%
% OUTPUTS:
%   a_E, a_W, a_N, a_S - Discretization coefficients for east, west,
   north, and south neighbors.
%   a_P                - Central coefficient of the discretized
   equation.
%   b_P                - Right-hand side term of the equation.
%-------------------------------------------------------

dx = (L/N);
dy = (H/M);

a_E = zeros(M+2,N+2);
a_W = zeros(M+2,N+2);
```

```
a_N = zeros(M+2,N+2);
a_S = zeros(M+2,N+2);
a_P = zeros(M+2,N+2);
b_P = zeros(M+2,N+2);

psi_bottom=psi_start;

for j = 2:M+1
    for i = 2:N+1
        a_E(j, i) = 1 * dy / (x_P(i+1)-x_P(i));
        a_W(j, i) = 1 * dy / (x_P(i)-x_P(i-1));
        a_N(j, i) = 1 * dx / (y_P(j+1)-y_P(j));
        a_S(j, i) = 1 * dx / (y_P(j)-y_P(j-1));
        a_P(j, i) = a_E(j, i) + a_W(j, i) + a_N(j, i) + a_S(j, i);
        b_P(j, i) = 0;
    end
end

% Bottom nodes
for i = 1:N+2
    a_E(1, i) = 0;
    a_W(1, i) = 0;
    a_N(1, i) = 0;
    a_S(1, i) = 0;
    a_P(1, i) = 1;
    b_P(1, i) = psi_bottom;
end

% Top nodes
for i = 1:N+2
    a_E(M+2, i) = 0;
    a_W(M+2, i) = 0;
    a_N(M+2, i) = 0;
    a_S(M+2, i) = 0;
    a_P(M+2, i) = 1;
    b_P(M+2, i) = psi_bottom + v_in * H;
end

% Inlet nodes
for j = 2:M+1
    a_E(j, 1) = 0;
    a_W(j, 1) = 0;
    a_N(j, 1) = 0;
    a_S(j, 1) = 0;
    a_P(j, 1) = 1;
    b_P(j, 1) = psi_bottom + v_in * y_P(j);
end

```

```
% Outlet nodes
for j = 2:M+1
    a_E(j, N+2) = 0;
    a_W(j, N+2) = 1;
    a_N(j, N+2) = 0;
    a_S(j, N+2) = 0;
    a_P(j, N+2) = 1;
    b_P(j, N+2) = 0;
end
% Solid nodes
for j = 2:M+1
    for i= 2:N+1
        if solid(j,i)==0
        a_E(j, i) = 0;
        a_W(j, i) = 0;
        a_N(j, i) = 0;
        a_S(j, i) = 0;
        a_P(j, i) = 1;
        b_P(j, i) = psi_body;
        end
    end
end
end
```

```

function [T_P, p_P, rho_P, cp_mean] = compute_thermo_properties(T_ref,
    p_ref, v_ref, v_P, R, gamma, M, N)

% COMPUTE_THERMO_PROPERTIES Computes thermodynamic properties from
    velocity and reference values.
%
% This function calculates the temperature, pressure, and density fields
    in a flow
% domain based on a reference temperature, pressure, and velocity.
%
% Inputs:
%   T_ref    - Reference temperature.
%   p_ref    - Reference pressure.
%   v_ref    - Reference velocity.
%   v_P      - Velocity magnitude field at each grid point.
%   R        - Specific gas constant.
%   gamma    - Heat capacity ratio (Cp/Cv).
%   M, N     - Number of grid points in the y and x directions.
%
% Outputs:
%   T_P      - Computed temperature field.
%   p_P      - Computed pressure field.
%   rho_P    - Computed density field.
```

```
%   cp_mean - Computed specific heat capacity field.
%
% A polynomial model is used to estimate the specific heat capacity as a function of temperature.
%-------------------------------------------------------

A = 1005;
B = 0.1;
C = -0.0002;

T_P = zeros(M+2, N+2);
p_P = zeros(M+2, N+2);
rho_P = zeros(M+2, N+2);

cp_mean = (1 ./ (T_P - T_ref)) .*(A .* (T_P - T_ref) +(B / 2) .* (T_P.^2 - T_ref^2) +(C / 3) .* (T_P.^3 - T_ref^3));

for i = 2:N+1
    for j = 2:M+1
        T_P(j,i) = T_ref + ((v_ref^2 - v_P(j,i)^2) / (2 * cp_mean(j,i)));
        p_P(j,i) = p_ref * (T_P(j,i) / T_ref)^(gamma / (gamma - 1));
        rho_P(j,i) = p_P(j,i) / (R * T_P(j,i));
        end
    end
end
```

```

function [v_xP, v_yP, v_P] = compute_velocity(psi, y_P, x_P, M, N, v_in, rho_ref, rho)

% This function calculates the velocity field in a computational domain based on the
% stream function   . It determines the velocity components in the x and y directions
% and computes the velocity magnitude at each node.
%
% INPUTS:
%   psi     - Stream function values at each computational node.
%   y_P     - Y-coordinates of the grid points.
%   x_P     - X-coordinates of the grid points.
%   M, N    - Number of nodes in the Y and X directions, respectively.
%   v_in    - Inlet velocity (free-stream velocity).
%   rho_ref - Reference density.
%   rho     - Density field
%
% OUTPUTS:
%   v_xP - Velocity component in the x-direction.
```

```
%   v_yP - Velocity component in the y-direction.
%   v_P  - Velocity magnitude at each node.
%-------------------------------------------------------

    v_xP = zeros(M+2, N+2);
    v_yP = zeros(M+2, N+2);

    rr_re = zeros(M+2, N+2);
    rr_rw = zeros(M+2, N+2);
    rr_rn = zeros(M+2, N+2);
    rr_rs = zeros(M+2, N+2);


% top and bottom walls     Tangencial velocity=0
for i = 1:N+2
    v_yP(1, i) = 0;
    v_yP(M+2, i) = 0;

end

% Developed flow at output
for j = 1:M+2
    v_xP(j, N+2) = v_xP(j, N+1);
    v_yP(j, N+2) = v_yP(j, N+1);
end

% Channel inlet (x = 0)     v_in
for j = 1:M+2
    v_xP(j, 1) = v_in;
    v_yP(j, 1) = 0;
end

for i = 2:N+1
    for j = 2:M+1

    %Blocking off method
    rr_re(j,i) = (x_P(i+1) - x_P(i)) / (((0.5 * (x_P(i+1) - x_P(i))) / (
        rho_ref / rho(j,i))) + ((0.5 * (x_P(i+1) - x_P(i))) / (rho_ref /
        rho(j,i+1))));
    rr_rw(j,i) = (x_P(i)-x_P(i-1)) / (((0.5 * (x_P(i)-x_P(i-1))) / (
        rho_ref / rho(j,i))) + ((0.5 * (x_P(i)-x_P(i-1))) / (rho_ref /
        rho(j,i-1))));
    rr_rn(j,i) = (y_P(j+1)-y_P(j)) / (((0.5 * (y_P(j+1)-y_P(j))) / (
        rho_ref / rho(j,i))) + ((0.5 * (y_P(j+1)-y_P(j))) / (rho_ref /
        rho(j+1,i))));
    rr_rs(j,i) = (y_P(j) - y_P(j-1)) / (((0.5 * (y_P(j) - y_P(j-1))) / (
        rho_ref / rho(j,i))) + ((0.5 * (y_P(j) - y_P(j-1))) / (rho_ref /
        rho(j-1,i))));
```

```
59
60      % Velocity at faces
61      v_xn(j, i) = +(1/(rr_rn(j,i))) * (psi(j+1, i) - psi(j, i)) / (y_P(j
            +1)-y_P(j));
62      v_xs(j, i) = +(1/(rr_rs(j,i))) * (psi(j, i) - psi(j-1, i)) / (y_P(j)
            -y_P(j-1));
63      v_ye(j, i) = -(1/(rr_re(j,i))) * (psi(j, i+1) - psi(j, i)) / (x_P(i
            +1)-x_P(i));
64      v_yw(j, i) = -(1/(rr_rw(j,i))) * (psi(j, i) - psi(j, i-1)) / (x_P(i)
            -x_P(i-1));
65
66      % Velocity components at nodes
67      v_xP(j, i) = (v_xn(j, i) + v_xs(j, i)) / 2;
68      v_yP(j, i) = (v_ye(j, i) + v_yw(j, i)) / 2;
69
70      end
71  end
72      % Velocity magnitude
73      v_P = sqrt(v_xP.^2 + v_yP.^2);
74  end
```

```
1
2   function [x_cyl, y_cyl, solid, theta] = cylinder_generator(x_center,
        y_center, R, M, N, x_P, y_P, solid)
3
4   % This function creates a set of points defining the perimeter of a
        cylinder
5   % centered at (x_center, y_center) with a radius R. Additionally, it
        updates
6   % the solid matrix to mark which grid points lie inside the cylinder.
7   %
8   % Inputs:
9   %   x_center, y_center - Coordinates of the cylinder center.
10  %   R - Radius of the cylinder.
11  %   M, N - Number of discretization points in the y and x directions.
12  %   x_P, y_P - Node points along the x and y directions.
13  %   solid - Predefined matrix indicating solid (0) or fluid (1) regions.
14  %
15  % Outputs:
16  %   x_cyl, y_cyl - Arrays containing the coordinates of the cylinder's
        perimeter.
17  %   solid - Updated matrix marking solid nodes inside the cylinder.
18  %
        ------------------------------------------------------------------------
19
20
21  theta = linspace(0, 2*pi, 10000);
```

```
x_cyl = x_center + R * cos(theta);
y_cyl = y_center + R * sin(theta);


for j = 1:M+2
    for i = 1:N+2
        if (x_P(i) - x_center)^2 + (y_P(j) - y_center)^2 <= R^2
            solid(j, i) = 0;
        end
    end
end
end
```

```

function [C_D, C_L, Gamma, D, L] = final_calculations(x_P, y_P, p_P,
    v_xP, v_yP, x_cyl, y_cyl, theta, R_cyl, w, rho_ref, v_in)

% This function calculates the aerodynamic properties of a rotating
    cylinder immersed in
% a potential flow. It determines the lift and drag forces, their
    respective coefficients,
% and the circulation around the cylinder.
%
% INPUTS:
%   x_P, y_P  - Grid point coordinates in the computational domain.
%   p_P       - Pressure field at each computational node.
%   v_xP, v_yP - Velocity components at each grid point.
%   x_cyl, y_cyl - Cylinder surface coordinates.
%   theta     - Angular positions along the cylinder surface.
%   R_cyl     - Radius of the cylinder.
%   w         - Angular velocity of the cylinder (rad/s).
%   rho_ref   - Reference density.
%   v_in      - Free-stream velocity.
%
% OUTPUTS:
%   C_D   - Drag coefficient.
%   C_L   - Lift coefficient.
%   Gamma - Circulation around the cylinder.
%   D     - Drag force.
%   L     - Lift force.
%-------------------------------------------------

% Interpolate pressure on the cylinder surface
p_cyl = interp2(x_P, y_P, p_P, x_cyl, y_cyl, 'spline');

% Interpolate velocity components at the cylinder surface
v_x_cyl = interp2(x_P, y_P, v_xP, x_cyl, y_cyl, 'spline');
v_y_cyl = interp2(x_P, y_P, v_yP, x_cyl, y_cyl, 'spline');
```

```
33
34 % Compute tangential velocity
35 v_theta = v_x_cyl .* cos(theta) + v_y_cyl .* sin(theta);
36
37 % Compute circulation
38 Gamma = 2 * pi * R_cyl^2 * w;
39
40 % Compute lift and drag forces
41 L = -trapz(theta, p_cyl .* sin(theta) * R_cyl);
42 D = -trapz(theta, p_cyl .* cos(theta) * R_cyl);
43 % Compute lift and drag coefficients
44 C_D = D / (0.5 * rho_ref * v_in^2 * (2 * R_cyl));
45 C_L = L / (0.5 * rho_ref * v_in^2 * (2 * R_cyl));
46
47 % Display results
48 fprintf('Lift Coefficient (C_L): %f\n', C_L);
49 fprintf('Drag Coefficient (C_D): %f\n', C_D);
50 fprintf('Circulation (Gamma): %f\n', Gamma);
51 end
```

```
1
2 function [psi, iter, error] = gauss_seidel(psi, psi_body, solid, a_E,
    a_W, a_N, a_S, a_P, b_P, v_in, y_P, M, N, H, delta)
3
4 % GAUSS_SEIDEL Solves the stream function equation using the Gauss-
    Seidel method.
5 %
6 % This function iteratively solves the discretized stream function
    equation
7 % using the Gauss-Seidel method with Dirichlet boundary conditions and
8 % internal conditions imposed by solid obstacles in the domain.
9 %
10 % Inputs:
11 %   psi       - Initial guess for the stream function.
12 %   psi_body  - Stream function value inside solid obstacles.
13 %   solid     - Binary matrix indicating solid (0) or fluid (1) regions.
14 %   a_E, a_W, a_N, a_S, a_P - Coefficients of the discretized equation.
15 %   b_P       - Right-hand side of the equation.
16 %   v_in      - Inlet velocity.
17 %   y_P       - Array of y-coordinates of the grid points.
18 %   M, N      - Number of grid points in the y and x directions.
19 %   H         - Channel height.
20 %   delta     - Convergence threshold for the iterative solver.
21 %
22 % Outputs:
23 %   psi       - Computed stream function field.
24 %   iter      - Number of iterations performed.
25 %   error     - Final error value.
```

```
%
% The solver iterates until convergence is achieved, applying boundary
% conditions at the channel walls and enforcing the presence of solid
% obstacles. The Gauss-Seidel method is used to update the stream
    function
% values at each iteration. The algorithm stops when the maximum
    difference
% between successive iterations is below the specified tolerance (delta)
    .
%-------------------------------------------------------

max_iter = 1000000;
error = Inf;
for iter = 1:max_iter
    psi_old = psi;
    % Channel enter (x = 0)
    for j = 1:M+2
        psi(j, 1) = v_in * y_P(j);
    end
    % Channel exit (x = L)
    for j = 1:M+2
        psi(j, N+2) = psi(j, N+1);
    end
    % Inferior wall
    for i = 1:N+2
        psi(1, i) = 0;
    end
    % Superior wall
    for i = 1:N+2
        psi(M+2, i) = v_in * H;
    end
    % Solid
    for j = 1:M+2
        for i = 1:N+2
            if solid(j, i) == 0
                psi(j, i) = psi_body;
            end
        end
    end

    for j = 2:M+1
        for i = 2:N+1
                psi(j, i) = (a_E(j, i) * psi(j, i+1) + a_W(j, i) * psi(j
                    , i-1) + ...
                                a_N(j, i) * psi(j+1, i) + a_S(j, i) * psi(j
                                    -1, i) + b_P(j, i)) / a_P(j, i);
        end
    end
```

```
    error = max(max(abs(psi - psi_old)));
    if error < delta
        break;
    end
end
end
```

```

function [x_P,y_P] = mesh_generator(N, M, L, H)

% MESH_GENERATOR Creates a structured Cartesian mesh for a 2D domain.
%
% This function generates a uniform computational grid for a domain
% of dimensions L    H, discretized with N points in the x-direction
% and M points in the y-direction.
%
% Inputs:
%   N, M - Number of grid points in the x and y directions, respectively
    .
%   L, H - Length and height of the domain.
%
% Outputs:
%   x_P - Array containing the x-coordinates of the cell centers and
    boundaries.
%   y_P - Array containing the y-coordinates of the cell centers and
    boundaries.
%
% The function initializes the mesh by defining the x and y coordinates
% of the grid points. It places the first and last points at the domain
% boundaries, ensuring proper discretization.

x_P=zeros(1, N+2);
x_P(2:end-1) = (0.5:1:N-0.5) * (L/N);
x_P(1) = 0;
x_P(end) = L;

y_P=zeros(1, M+2);
y_P(2:end-1) = (0.5:1:M-0.5) * (H/M);
y_P(1) = 0;
y_P(end) = H;

end
```

```

function mesh_plot(x_P, y_P, L, H, N, M)

% INPUTS:
```

```
%   x_P - X-coordinates of the mesh nodes.
%   y_P - Y-coordinates of the mesh nodes.
%   L   - Length of the computational domain.
%   H   - Height of the computational domain.
%   N   - Number of nodes in the X-direction.
%   M   - Number of nodes in the Y-direction.
%
% OUTPUT:
%   A 2D plot displaying the mesh structure, including node positions,
%   cell divisions, and domain boundaries.
%-------------------------------------------------------

dx = L/N;
dy = H/M;

[X, Y] = meshgrid(x_P, y_P);

% Plot the nodes
plot(X(:), Y(:), 'b.', 'MarkerSize', 0.5, 'MarkerFaceColor', 'b');
axis equal
axis tight
hold on;

for i = 2:N
    plot([x_P(i)+0.5*dx, x_P(i)+0.5*dx], [0, H], 'k--', 'LineWidth',
        0.1);
end

% Horizontal lines
for j = 2:M
    plot([0, L], [y_P(j)+0.5*dy, y_P(j)+0.5*dy], 'k--', 'LineWidth',
        0.1);
end

%Channel walls
plot([0, L], [0, 0], 'k-', 'LineWidth', 1); % L nea inferior en negro y
    m s gruesa
plot([0, L], [H, H], 'k-', 'LineWidth', 1); % L nea superior en negro y
    m s gruesa
plot([0, 0], [0, H], 'k--', 'LineWidth', 1); % L nea izquierda
plot([L, L], [0, H], 'k--', 'LineWidth', 1); % L nea derecha
 axis equal
 axis tight
title('Malla de c lculo con nodos en el centro de las celdas');
xlabel('x [m]'); ylabel('y [m]'); grid off;

end
```

```

function plotCFD(titleStr, F, N, M, dx, dy)
% plotCFD generates a graphical representation of the scalar field F using a color map.
%
% Inputs:
%   titleStr - Title of the graph (string).
%   F        - Matrix representing the scalar field to be plotted.
%   N        - Number of discretization points in the x-direction.
%   M        - Number of discretization points in the y-direction.
%   dx       - Grid spacing in the x-direction.
%   dy       - Grid spacing in the y-direction.
%
% Outputs:
%   A graphical representation of F with color coding and a color bar.
%-------------------------------------------------------

% Plot the field F
imagesc((0.5:N-0.5) * dx, (0.5:M-0.5) * dy, F(2:end-1, 2:end-1));
axis xy equal tight;
colorbar;
shading interp;
colormap jet;

% Set axis labels
xlabel('x [m]');
ylabel('y [m]');
grid off;

% Set the title correctly
if isempty(titleStr)
    title('Function F', 'FontWeight', 'bold', 'FontSize', 12);
else
    title(['Function ', titleStr], 'FontWeight', 'bold', 'FontSize', 12);
end

end
```

```

function plotStreamLines(psi, x_P, y_P)

% PLOTSTREAMLINES Plots the streamlines of a flow field using contour lines.
%
% This function generates a streamline plot by contouring the stream function
```

```
% ‘psi‘ over a structured grid defined by ‘x_P‘ and ‘y_P‘. The function
% represents streamlines that indicate the direction of the flow.
%
% Inputs:
%   psi  - Matrix of stream function values.
%   x_P  - Vector of x-coordinates for the grid points.
%   y_P  - Vector of y-coordinates for the grid points.
%
% The function uses ‘contour‘ to draw 30 evenly spaced streamlines with
% black color and a thin line width. The contour lines represent the
   flow
% paths in a 2D domain.
%
% Outputs:
%   A graphical representation of streamlines.
%-------------------------------------------------------

[X, Y] = meshgrid(x_P, y_P);

contour(X, Y, psi, 30, 'k', 'LineWidth', 0.01);

end
```